4.h Sealed Classes & Interfaces

Controlled Inheritance in Kotlin

Contents

1 **Sealed Classes**
Definition, syntax, constructors & inheritance rules

2 **Sealed Interfaces**
Multiple inheritance, flexibility & comparison with sealed classes

3 **when Expressions**
Exhaustive checks, no else branch & compiler safety

4 **Use Cases & Best Practices**
State management, API responses, sealed vs enum classes

What Are Sealed Classes?

Sealed classes and interfaces provide **controlled inheritance** of your class hierarchies. With it, all direct subclasses are known at compile time.

Key characteristics of sealed classes:

Abstract by Default

A sealed class is always abstract and cannot be instantiated directly. Constructors are protected or private.

Restricted Hierarchy

Direct subclasses must be in the same package. No external subclasses after the module is compiled.

Exhaustive when

The compiler verifies all possible cases are covered, eliminating the need for an else branch.

Flexible Subclasses

Subclasses can be data classes, objects, regular classes, or even other sealed classes.

Declaring a Sealed Class

Syntax

```
sealed class Error(val message: String) {
    class NetworkError(msg: String) : Error(msg)
    class DatabaseError(msg: String): Error(msg)
    class UnknownError        : Error("Unknown")
}
```

Constructor Rules

- Constructors are protected, or private by default
- Cannot have public or internal constructors
- The sealed class itself cannot be instantiated — only its subclasses
- Subclasses call the parent constructor as usual

💡 A sealed class can have fields and methods (both abstract and implemented), just like an abstract class.

Inheritance Rules

Direct subclasses of sealed classes and interfaces must be declared in the same package.

Direct Subclasses

- Must be in the same package
- Can be top-level or nested
- Must have a qualified name
- Can't be local or anonymous

Indirect Subclasses

- No restrictions on package
- Can be defined anywhere
- Can extend direct subclasses freely in other modules

Visibility

Subclasses can have any visibility compatible with Kotlin inheritance.

Enum Restriction

enum classes can't extend a sealed class but can implement sealed interfaces.

Sealed Classes with when Expressions

The when expression allows the Kotlin compiler to check exhaustively that all possible cases are covered.

```kotlin
sealed class Result {
        data class Success(val data: String) : Result()
        data class Error(val msg: String) : Result()
        object Loading : Result()
    }

    fun handle(result: Result) = when (result) {
        is Result.Success -> show(result.data)
        is Result.Error -> show(result.msg)
        Result.Loading -> showLoadingInfo()
        // no else needed!
    }
}

fun show(msg: String) { println(msg) }
fun showLoadingInfo() { println("Loading...") }
```

No else Needed

Compiler knows all subclasses, so all branches are verified.

Compile-Time Safety

Adding a new subclass triggers a compiler error if unhandled.

Guard Conditions

You can add extra conditions in a single when branch.

Sealed Interfaces

Sealed interfaces work like sealed classes but allow a class to implement multiple sealed hierarchies.

```
sealed interface Error2 { val message: String }

sealed interface Recoverable

class NetworkError(override val message: String) : Error2, Recoverable

class DatabaseError(override val message: String) : Error2

enum class IoError : Error2 {
    FileNotFound, PermissionDenied;

    override val message: String
        get() = when (this) {
            FileNotFound -> "File not found"
            PermissionDenied -> "Permission denied"
        }
}

val errors: List<Error2> = listOf(
    NetworkError("Temporary network issue"),
    DatabaseError("Schema mismatch"),
    IoError.FileNotFound,
    IoError.PermissionDenied
)
```

Multiple Inheritance

A class can implement multiple sealed interfaces — impossible with sealed classes.

Enum Compatibility

enum classes can implement sealed interfaces, but cannot extend sealed classes.

💡 Sealed interfaces cannot have constructors, but can have properties and default method implementations.

Sealed Classes vs Enum Classes

Feature	Enum Class	Sealed Class
Instances	Fixed singletons	Multiple instances per subclass
Data per variant	Same structure for all	Different data per subclass
Inheritance	Cannot extend classes	Subclasses extend sealed class
Use when	Simple flags / constants	States with different payloads

Use Enum When...

Options only differ in meaning, not structure. (e.g., status flags like ACTIVE, INACTIVE, PENDING)

Use Sealed When...

Each variant carries different data or behavior. (e.g., Success with data, Error with exception, Loading)

Use Case: UI State Management

Sealed classes are ideal for representing different UI states — structured, safe handling of UI changes.

```
sealed class UiState<out T> {
    object Loading : UiState<Nothing>()
    data class Success<T>( val data: T ) : UiState<T>()
    data class Error( val exception: Throwable ) : UiState<Nothing>()
}


fun render(state: UiState<String>) {
    when (state) {
        is UiState.Loading -> showLoadingInfo()
        is UiState.Success -> show(state.data)
        is UiState.Error -> showError(state)
    }
}


fun showError(state: UiState.Error) { println(state) }
…
```

Why Sealed Classes Shine Here

- Each state carries different data (Success has T, Error has Throwable)
- Compiler enforces all states are handled in when expression
- Adding a new state (e.g., Empty) causes compile errors where unhandled — no runtime surprises
- Works with generics for reusable state containers across the app

Use Case: Payment Processing

Sealed classes with when expressions let you implement business logic where each variant has specific properties.

```
sealed class Payment {
    data class CreditCard( val number: String, val expiryDate: String)
                                                : Payment()
    data class PayPal( val email: String ) : Payment()
    object Cash : Payment()
}

fun processPayment(p: Payment) =
    when (p) {
        is Payment.CreditCard -> chargeCard(p.number, p.expiryDate)
        is Payment.PayPal -> chargePayPal(p.email)
        Payment.Cash -> acceptCash()
    }
```

Pattern Highlights

- Each payment type has unique properties (number, email, etc.)
- Smart casts grant direct access to subclass-specific properties
- Adding a new payment method (e.g., Crypto) forces updating every when expression
- Type-safe: impossible to forget handling a payment variant

Best Practices & Summary

Use data classes

Prefer data classes for leaf nodes in the sealed hierarchy to get equals(), hashCode() and copy() for free.

Use object for states

Use object declarations for stateless variants like Loading or Idle — they're singletons and save memory.

Avoid else branch

Never add an else branch with sealed when — you lose compile-time safety when new subclasses are added.

Sealed interface

Choose sealed interface over sealed class when you need multiple inheritance or flexible hierarchies.

Combine with generics

Type parameters make sealed hierarchies reusable across domains: Resource<T>, UiState<T>, Response<T>.

Nest sealed classes

Nest sealed classes within each other for hierarchical or multi-level state structures (e.g., Success > Data, Empty).